AI-Native 開發工作坊

YTP2026H2 E2
蔡芸琤 Yun-Cheng Tsai
2026.10.17 09:00–13:00（台灣時間）

VS Code 實作
Colab 重現
同一份程式，同一組驗收

檢查點 今天的成果：一個可解釋、可修改、可驗證的變更

今天要交出的成果

01 找到「記得」按鈕的資料流
02 新增三個程式主題單字
03 新增 countDue 與測試
04 在 Colab 重做並比較結果

新增單字
loop＝迴圈
variable＝變數
debug＝除錯

新增純函式
countDue(states, today)

檢查點 能說清楚每一個改動的理由

四小時操作節奏

09:00–09:20 目標、環境與起點
09:20–09:50 三輪提問與抽驗
09:50–10:30 指示檔與單字修改
10:30–10:40 休息
10:40–11:20 countDue 與驗收
11:20–11:30 休息
11:30–12:20 Colab 重做
12:20–12:45 技能與 review
12:45–13:00 展示與收尾

檢查點 提前完成者：請同伴解釋測試與 diff

課前環境檢查

VS Code、Git、Node.js、Python
一個能讀取工作資料夾的 AI 工具
Google 帳號與 Colab
下載操作包並解壓縮

```
git --version
node --version
python --version
```

macOS 可用 python3
Windows 可用 py

檢查點 版本能顯示，工具能開啟資料夾

教材、完成品與本次起點

線上教材：自學主線與 Lab 規格
官方 repo：完成品、驗收與歷史
操作包 starter：v1.1 的 lab10 快照
solution：本次雙環境任務解答

教材
tw-ytp.github.io/vocabuddy/

參考倉庫
github.com/ytp26h2/ytp26h2-labs

檢查點 先自行修改，再對照解答

同一個任務，兩種工作環境

VS Code
AI 讀取本機檔案，提出修改
終端機跑 JavaScript 測試
瀏覽器檢查 UI 與 localStorage

Colab
儲存格顯示檔案，再提供給 AI
把回傳程式寫回 app/
同樣用 Node 跑測試
下載結果保存

檢查點 兩邊共用資料與驗收，不假設工具能力相同

VS Code 01：開啟操作包

File → Open Folder → starter
確認左側有 app/ 與 PROJECT_RULES.md
Terminal → New Terminal
終端機目前位置應為 starter

```
starter/
  PROJECT_RULES.md
  app/
    index.html
    words.js
    srs.js
    test_srs.js
```

檢查點 AI 工作目錄與終端機指向同一份 starter

VS Code 02：建立自己的變更歷史

在 starter 建 Git repo
第一次 commit 留下未修改的起點
建立練習分支

```
git init
git add .
git commit -m "chore: E2 baseline"
git switch -c e2/practice
```

檢查點 git status 顯示目前分支 e2/practice

VS Code 03：先跑原本的測試

在終端機進入 app
逐一執行，不忽略報錯
保存原始輸出作為基準

```
cd app
node test_srs.js
node test_csv.js
node check_words.js
```

檢查點 三支原始測試都通過

VS Code 04：打開可操作的畫面

終端機停留 app
啟動 HTTP 伺服器
瀏覽器開 localhost:8000
按「顯示答案」，再按「記得」

```
python -m http.server 8000
```

替代命令

```
python3 -m http.server 8000
py -m http.server 8000
```

檢查點 畫面能換卡，Console 沒有載入錯誤

Lab 11：確認 AI 真的讀到檔案

先唯讀，檢查實際檔案
工具應指出單字所在的位置
自己打開 words.js 抽驗

只根據 starter 的檔案回答：
這個專案做什麼？
主要檔案各負責什麼？
單字在哪個檔案、哪個變數？
目前有幾個單字？請附證據。

檢查點 答案指向 app/words.js 的 WORDS

第一輪：專案地圖

只看 app/，避免混入不同 Lab
找出 HTML 載入哪些 JS
分清畫面、單字與純函式

只根據目前資料夾回答，不要修改。
列出 app/ 的主要檔案與責任、
載入關係與測試入口。
每個判斷附檔名與行號。
請區分觀察與推論。

檢查點 能指出 index.html 載入 words、srs、csv

第二輪：按下「記得」之後

只追這一個事件
每一步都打開程式碼核對
留意 queue 更新也在流程中

追蹤 app/index.html 的 mark(true)。
資料怎樣經過 schedule、
寫回學習紀錄、save、佇列與 next？
每一步請附檔名與行號。

檢查點 stateOf、schedule、save、queue、next 都有證據

第三輪：為什麼傳入 today

排程函式接收數值日期
固定 today 可重現測試
代價是呼叫者要提供正確單位

為什麼 schedule(box, ok, today)
由外部傳入 today？
用 srs.js 與 test_srs.js 的證據回答。
程式裡的 today 以什麼單位表示？

檢查點 能用 today=100 解釋 due=102

行號抽驗與錯誤修正

選 AI 回答中的兩個行號
Ctrl+G 跳到原始碼
核對函式、條件與實際資料
記下一條更正後的說法

常見錯誤
把不同 Lab 的行號混在一起
憑空說有後端資料庫
漏掉 queue 的更新
把天數誤當毫秒

檢查點 每個關鍵結論至少有一項可開啟的證據

Lab 12：一份可驗證的專案規則

補齊 PROJECT_RULES.md
記錄執行命令與測試命令
限制修改範圍與不自動推送
固定記號：// 用途：

這是什麼專案
怎麼啟動與測試
檔案與程式慣例
每次修改的驗收流程
不能做的事

檢查點 規則描述的是本次 starter/app 的實際情況

指示檔對照實驗

A：新對話，不額外提供規則
B：相同起點，新對話，提供規則
兩邊提出相同的新函式請求
比較固定記號與測試是否出現

請新增純函式 countDue 的計畫。
先不要改檔。
說明你會讀哪些檔案、
函式上方會放什麼註解、
以及修改後會跑什麼測試。

檢查點 只比較規則是否遵守，不預設 B 一定更好

第一個修改：三個單字

先計畫，再回覆「執行」
只動 app/words.js
保留所有既有資料與匯出
加過的字不重複

新增 loop=迴圈、variable=變數、
debug=除錯，已有就跳過。
先讀 words.js 與 check_words.js。
先列計畫，等我確認才改。
改完執行 node check_words.js。

檢查點 這份起點由18字變成21字

單字修改的驗收

自己開 diff 看三筆資料
跑 check_words.js
重新整理畫面與檢查 Console
已快取時清除 SW/cache 再重載

在 app/ 執行
node check_words.js

在 starter/ 執行
git diff -- app/words.js

檢查點 只有預期單字改動，原本單字都保留

休息 10 分鐘

10:30–10:40
保留目前程式與測試輸出
未完成者請助教看完整錯誤訊息

下一段
新增可測試的到期計數
一個條件，一組邊界案例

檢查點 回來時終端機與 VS Code 都留在 starter

第二個修改：countDue 規格

輸入：states，每筆有數值 due
輸入：today，有限數值
輸出：due <= today 的筆數
不改輸入，不讀時鐘

```
today = 100

states = [
  { due: 99 },
  { due: 100 },
  { due: 101 }
]
```

預期回傳：2

檢查點 今天到期也要算，條件是 <=

countDue 的修改計畫

只新增函式與匯出
保留 schedule 與 INTERVALS
新增 test_count_due.js
暫不接 UI

請在 app/srs.js 新增 countDue。
只接收 states 與 today，計算 due <= today。
不改輸入、不讀時鐘、不改 schedule。
每個新函式前加 // 用途：。
請先列計畫與測試案例，等我確認。

檢查點 能用自己的話說明預計修改的兩個檔案

邊界測試的預期結果

空陣列：0
99、100、101；today=100：2
101、102；today=100：0
重跑相同輸入：結果相同
呼叫前後：輸入保持不變

```
const { countDue } = require("./srs.js");

countDue([], 100)
// 0

countDue([{due:100}], 100)
// 1
```

檢查點 先寫下預期結果，再執行程式

原始測試與新測試一起跑

四支測試依序執行
失敗時保留完整輸出
用報錯位置查證原因
不刪測試換取通過

在 app/ 執行
node test_srs.js
node test_csv.js
node check_words.js
node test_count_due.js

再跑操作包提供的 e2_contract.js

檢查點 新功能通過，也保留原始行為

diff 包含什麼，review 才看得到什麼

git diff 只看尚未暫存的追蹤檔
新檔要從 git status 找到
暫存指定檔後再看完整 diff
commit 留到 review 完成後

在 starter/ 執行
git status
git diff
git add app/words.js app/srs.js
app/test_count_due.js
git diff --cached

檢查點 完整 diff 含新增函式與新增測試

休息與環境切換

11:20–11:30
VS Code 成果保留
接著重新使用原始起點
在 Colab 重做相同任務

下載 YTP_E2_Colab.ipynb
開啟 colab.research.google.com
檔案 → 上傳筆記本
檔案 → 在雲端硬碟中儲存副本

檢查點 CPU 即可，不需 GPU 或 API key

Colab 01：還原相同起點

執行第一個程式格
Notebook 已內嵌 v1.1 程式快照
讀取 /content/ytp-e2/app
印出來源 commit 與檔案列表

BASE：Colab VM 的練習資料夾
APP：要修改的 JavaScript 程式

重跑初始化會還原 app/
完成後先下載，才重跑

檢查點 確認 words.js 仍是未修改的18字起點

Colab 02：跑同一組原始測試

執行 Node 環境檢查格
缺少 Node 時，格子會安裝
run_node 會顯示 stdout/stderr
非零結束碼會停止該格

```
for name in [
    "test_srs.js",
    "test_csv.js",
    "check_words.js"
]:
    run_node(name)
```

檢查點 三支測試通過，字數與 VS Code 起點一致

Colab 03：提供可查證的上下文

執行帶行號的程式輸出格
把輸出貼給可用的 AI 對話
再貼相同的地圖、事件與設計問題
抽驗它指出的兩個行號

檔案更新後，再輸出一次
AI 看到的是你提供的快照

Notebook 的 Python 可以讀檔
AI 是否能讀檔要個別確認

檢查點 AI 回答能連回這次輸出的實際程式碼

Colab 04：再新增三個單字

從筆記本複製 words 提示詞
先確認計畫，再讓 AI 產出完整檔案
貼入 updated_words 的三引號內
執行寫檔與 check_words.js

```
updated_words = r"""完整 words.js"""

(APP / "words.js").write_text(
    updated_words, encoding="utf-8"
)
run_node("check_words.js")
```

檢查點 只有三個新增字，總數21，沒有重複

Colab 05：再實作 countDue

使用完全相同規格
貼完整 srs.js 與新測試檔
執行四支測試
再執行教師提供的合約測試

必須保留
schedule 的行為與原始匯出

必須新增
countDue(states, today)
test_count_due.js

檢查點 due=100、today=100 必須算一筆

Colab 06：diff 與保存

執行 difflib 比較格
檢查 words.js、srs.js 與新測試
執行下載成果格
保存 notebook 的副本

下載內容
JavaScript 程式
測試與合約驗收

Notebook 與 app 壓縮檔
兩份都要保存

檢查點 下載的成果可回 VS Code 以 HTTP 啟動

兩環境的結果比較

相同：原始程式、任務與合約測試
可能不同：AI 回答、改法與上下文
VS Code：代理讀寫、HTTP UI
Colab：儲存格、檔案快照、Node

請各組回答
哪一項規格兩邊都通過？
哪一次 AI 回答需要補上下文？
哪個測試抓到真實問題？
哪些 UI 行為仍需人工驗證？

檢查點 比較實際證據，不只比較「能跑」

Lab 13：新增單字的可重用流程

觸發條件：新增或修改單字包
先讀格式與既有資料
新增前去重並保留匯出
修改後執行檢查器
最後看 diff，由人決定 commit

正向測試
「新增三個天氣主題單字」
應讀單字資料並驗證

負向測試
「卡片圓角改大」
不應修改單字包

檢查點 流程能重用，且不會套到不相關任務

規則、技能與 reviewer 的分工

專案規則：每次改動共同遵守
技能：特定任務才採用的操作流程
reviewer：獨立檢查變更與證據

本堂共同規則
新函式加 // 用途：

本堂新增單字流程
讀格式、去重、修改、測試

本堂 reviewer
只讀 diff，不修改

檢查點 能說明三種內容各適合放什麼資訊

Lab 14：人先審，再請 AI 審

先記錄自己的判斷
主對話先取得完整 diff
把 diff、新檔與測試輸出交 reviewer
有子代理功能時用唯讀角色
無此功能則開新對話做第二次審查

請只讀以下變更，檢查：
需求符合、邊界、回歸、額外改動。
每項附嚴重度、檔名與行號、
重現案例與建議。
不要修改，不要把未跑寫成通過。

檢查點 審查者收到完整證據，人負責最後決定

人審與 AI 審查的對照紀錄

共同發現：兩邊都指出的問題
只有人發現：AI 漏掉的地方
只有 AI 發現：人要再查證
AI 誤報：附反證與不採納原因

例：AI 說 < 和 <= 都可以
反證：due=100，today=100
預期：1，使用 < 得到0

測試輸出與需求能支持判斷

檢查點 至少留下「一項發現」或「未發現問題的依據」

課後交付與三分鐘展示

展示按鈕事件與對應程式
說明新增功能及一個邊界案例
提供 diff 與實際測試輸出
附兩環境比較與 review 決定

建議繳交
自己的 repo 或程式壓縮檔
Colab notebook 與下載成果
PROMPTS、計畫與驗收紀錄

繳交位置與期限由主辦另行公告

檢查點 每個人都能解釋自己採納的修改

常見卡點與處理方式

讀不到檔案：確認工作目錄或提供全文
找不到 node：確認安裝與終端機重開
匯出錯誤：保留原本 module.exports
新測試沒出現在 diff：查看 git status

畫面仍舊版
清除 service worker/cache 後重載

Colab 重新連線後檔案不見
重新初始化，或上傳已下載成果

AI 額度不足
依規格手動實作，再跑同樣測試

檢查點 求助時附命令、工作目錄、完整錯誤與預期

課前演練與參考資料

教師：完整跑一次 starter 與 notebook
助教：Windows/macOS 各驗一次
備援：solution、完整提示詞與測試
官方 Lab 11–14 供課後接續

本包來源
四份 YTP 附件（2026.10.15）
ytp26h2-labs 的 v1.1 / lab10

原教材
https://tw-ytp.github.io/vocabuddy/
Colab FAQ
https://research.google.com/colaboratory/faq.html

檢查點 課堂主線完成後，再依原教材延伸